

Compiler Principle

Text Book: Modern Compiler Implementation in C

Author: Andrew W. Appel

2025 Spring&Summer

General Course Information

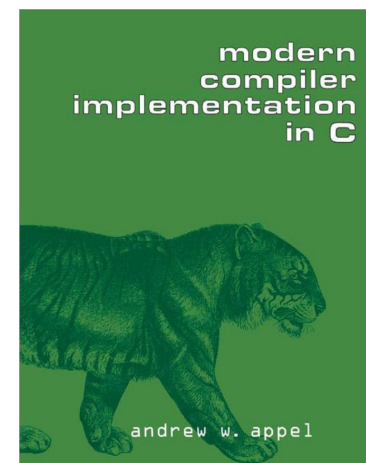
Teacher	Liu, Zhongxin (刘忠鑫), liu_zx@zju.edu.cn 研究方向：智能化软件工程，自动编程等
TAs	负责实验答疑、验收、实验报告批改： Yang, Peishan (杨沛山), yangps@zju.edu.cn , Deng Minghui (邓铭辉), 3210105650@zju.edu.cn 负责理论答疑与理论作业批改： Wang, You (王优), 3200102524@zju.edu.cn Wang, Junyi (王俊怡), wangjunyi@zju.edu.cn

Textbook:

<Modern Compiler Implementation in C>,
Andrew W. Appel

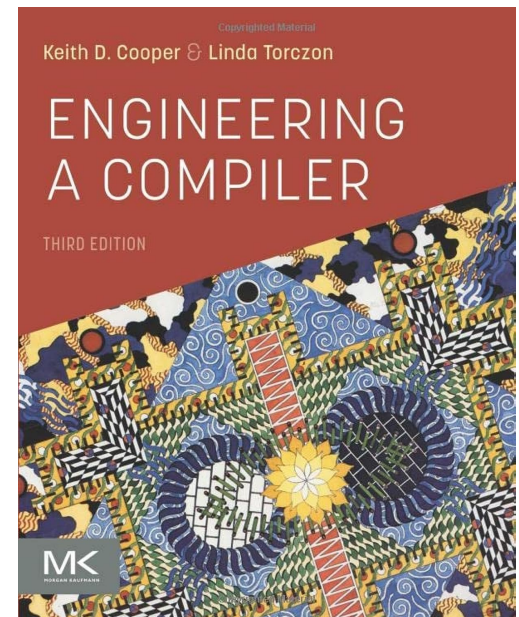
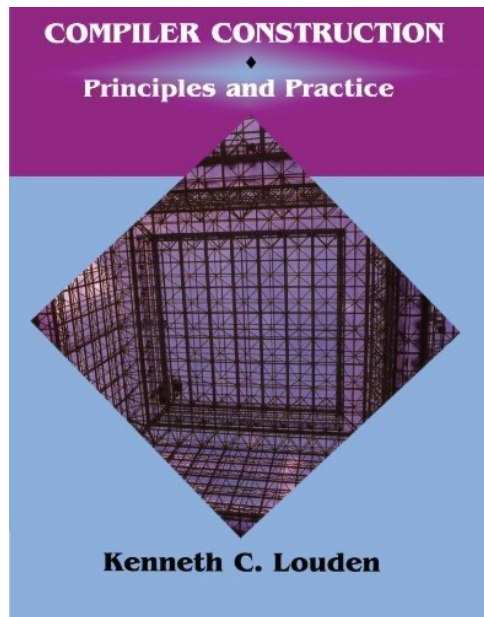
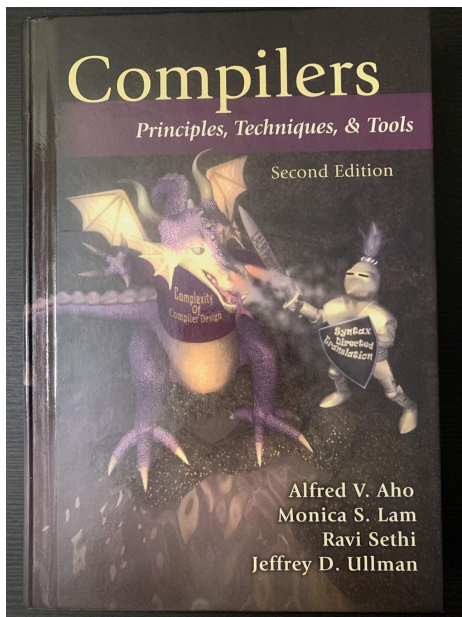
Course Materials:

学在浙大, <https://compiler.pages.zjusct.io/sp25/>



Reference Book

- ***Compilers -- Principles, Techniques and Tools***, (Dragon Book), by Aho, Sethi and Ullman
- ***Compiler Construction Principles and Practice***, by Kenneth C. Louden
- ***Engineering a Compiler***, by Keith D. Cooper, Linda Torczon



Grading Policies

- Homework = 10%
 - 10次，学在浙大上布置和提交
- Class Quizzes = 10%
 - 4次左右，理论课上随堂测试
- Mid Exam = 15%
 - 暂定第九周理论课上进行
- Labs = 25%
 - 由5个实验构成
- Final Exam = 40%

Final Exam < 40/100 → Final Grade < 60/100

- DDL、习题课和实验讲解的安排见钉钉群中的教学日历。

How to Survive!

- Homework = 10% → 一定要交!
- Class Quizzes = 10% → 记得来!
- Mid Exam = 15% → 努力复习拿高分。
- Labs = 25% → 布置后马上开始，多花时间，积极向助教寻求帮助，不要放弃。
- Final Exam = 40% → 平时分拿到40分以上，期末压力会小非常多。

Final Exam < 40/100 → Final Grade < 60/100

Outline

- **What, Why and How?**

- What is a compiler?
- Why do we need to learn compilers?
- How to learn compilers?

- **Introduction**

- Modules and Interfaces
- Tools and Software
- Data Structures for tree languages

What is a Compiler?

- A compiler is a program that can translate a program in one language – the **source** language – into an **equivalent** program in another language – the **target** language.



- A compiler is a **complex program**
 - From 10,000 to 1,000,000 lines of codes

Why Should We Learn Compilers?

- Compilers are used in almost all forms of computing.
- The principles and techniques are applicable to so many other domains.
 - E.g., command interpreters and interface programs
 - Help understand programming and computing
- Compiler writing touches upon other fields:
 - programming languages
 - computer architecture
 - algorithms
 - software engineering.

How to Learn Compilers?

- **Lectures** describe:
 - **Techniques**
 - **Data Structures**
 - **Algorithm**

for translating programming languages into executable code

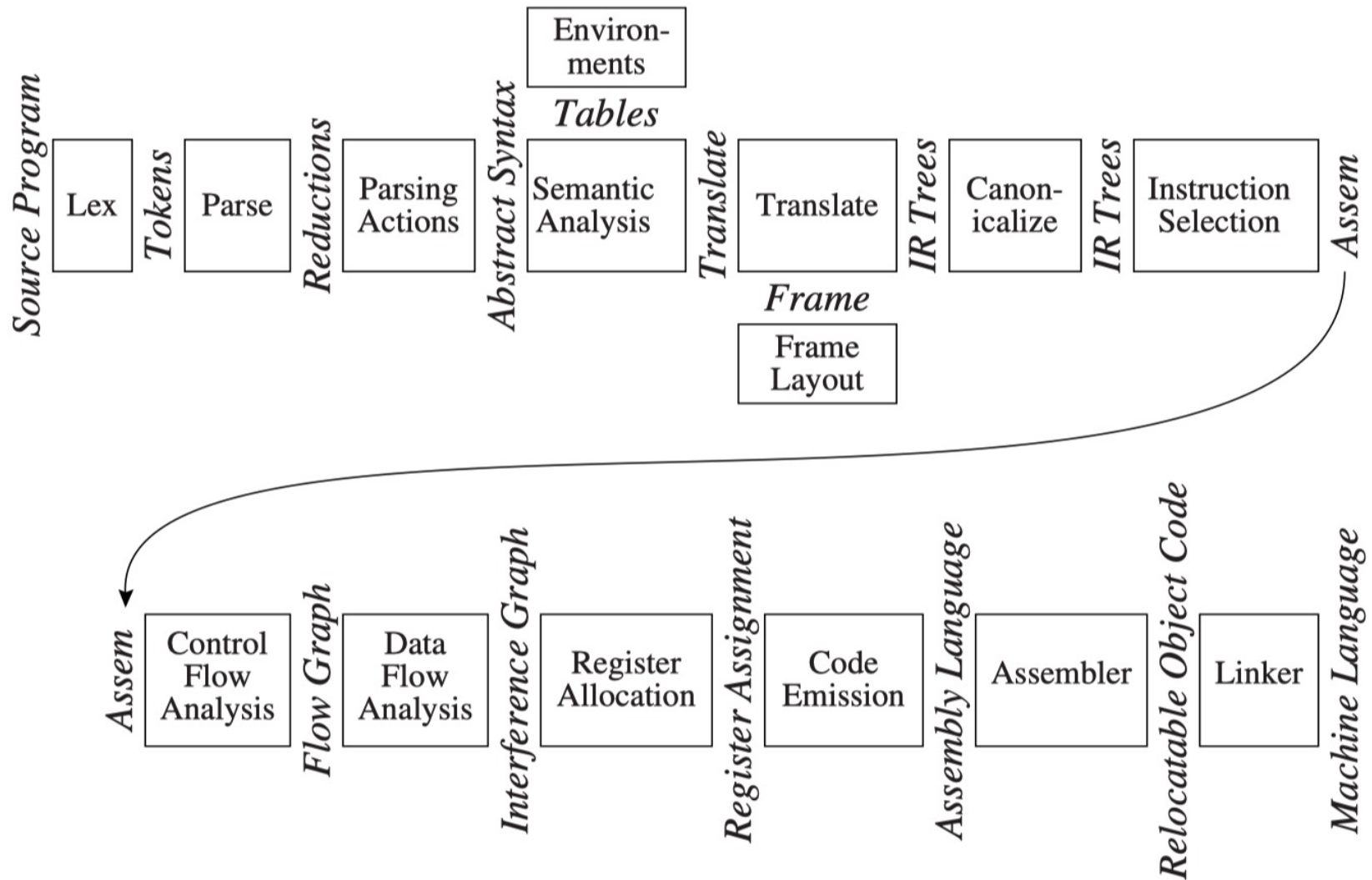
- **Labs** aim to implement a compiler: SysY $\rightarrow$ RISC-V
- **The Textbook:** show how to compile **Tiger** programming language in *C programming language*
 - Gaps between the textbook and the labs
 - Appendix A describes **Tiger**

Outline

- What, Why and How?
 - What is a compiler?
 - Why do we need to learn compilers?
 - How to learn compilers?
- Introduction
 - Modules and Interfaces
 - Tools and Software
 - Data Structures for tree languages

1.1 Modules and Interfaces

Modules and Interfaces: Overview



The phases, interfaces in a typical compiler

Two Important Concepts

Phases: one or more modules

Operating on the different abstract “*languages*”
during compiling process

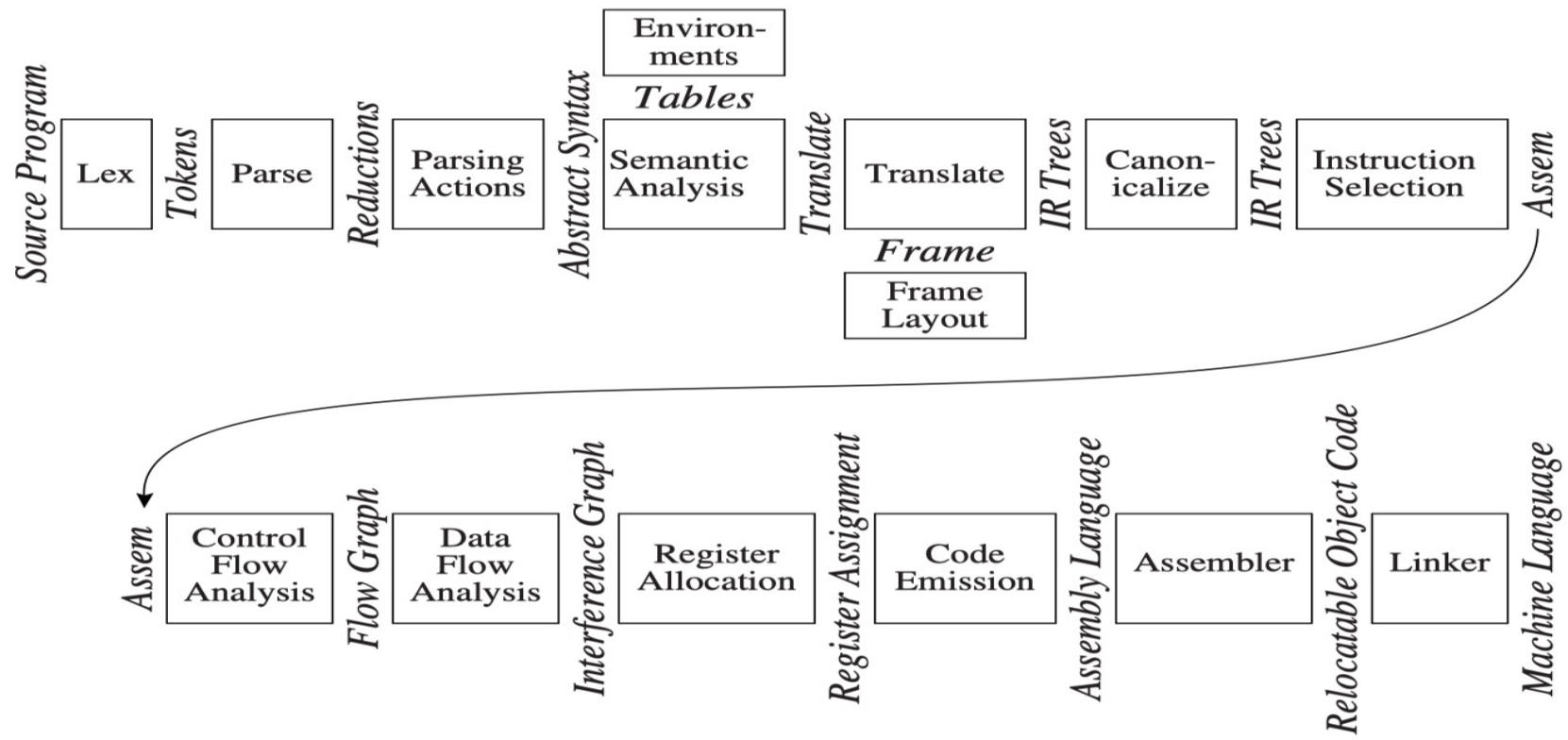
Interfaces:

Describe the information exchanged between
modules of the compiler

Module and Interfaces: Phases

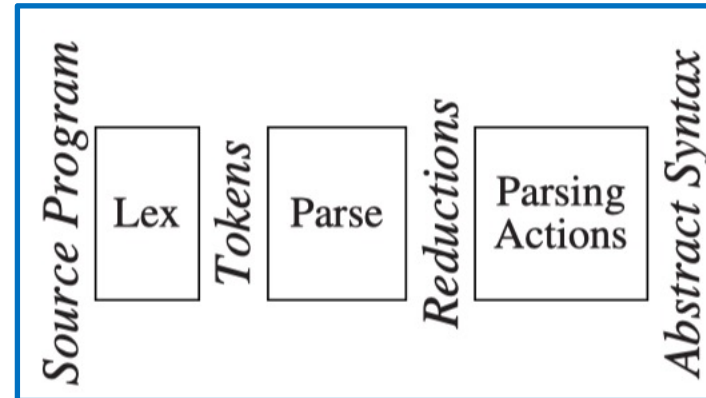
Why break the compiler into multiple phases?

- Easier to understand and implement
- Allowing for **reuse of the components**
 - Changing the source language
 - Used in an IDE



Module and Interfaces: Phases

Description of compiler phases

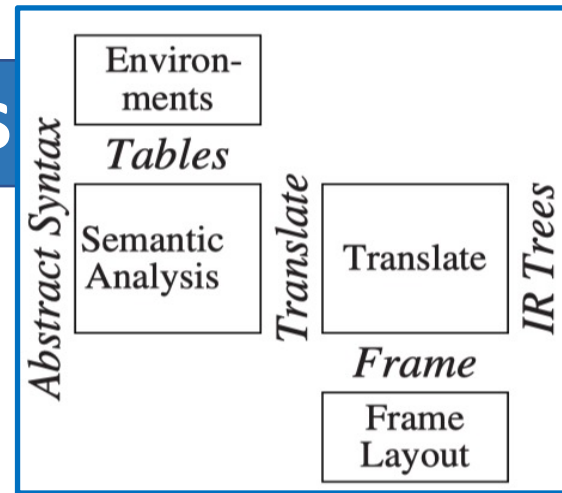


Chapter	Phase	Description
2	Lex	Break the source file into individual words, or <i>tokens</i>
3	Parse	Analyze the phrase structure of the program
4	Parsing Actions	Build a piece of <i>abstract syntax tree</i> corresponding to each phrase

Modules and Interfaces: Phases

Description of compiler phases

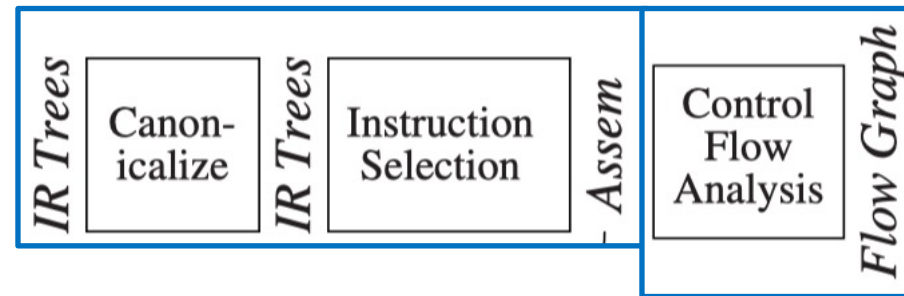
Chapter	Phase	Description
5	Semantic Analysis	Determine what each phrase means Relate uses of variables to their definitions Check types of expressions Request translation of each phrase
6	Frame Layout	Place variables, function-parameters, etc. into activation records (stack frames) in a machine-dependent way.
7	Translate	Produce intermediate representation trees (IR trees) Not tied to any particular source language or target-machine architecture.



Modules and Interfaces: Phases

Description of compiler phases

Chapter	Phase	Description
---------	-------	-------------



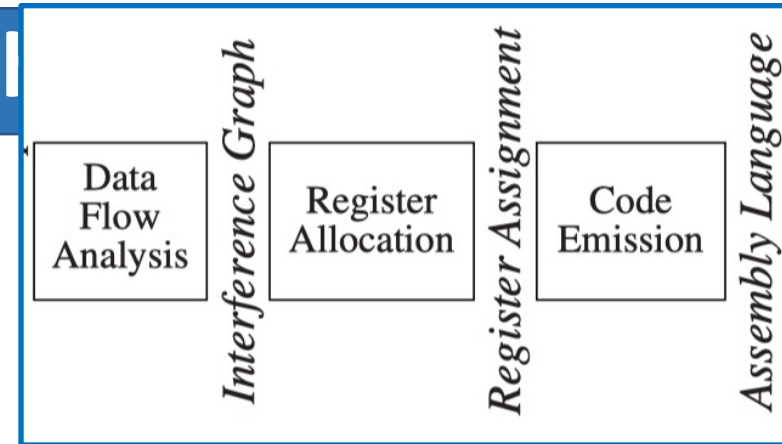
- | | | |
|----|-----------------------|---|
| 8 | Canonicalize | Hoist side effects out of expressions
Clean up conditional branches
for the convenience of the next phases. |
| 9 | Instruction Selection | Group the IR-tree nodes into clumps that correspond to the actions of target-machine instructions. |
| 10 | Control Flow Analysis | Analyze the sequence of instructions into a control flow graph that shows all the possible flows of control the program might follow when it executes. |

Modules and Interfaces: Pl

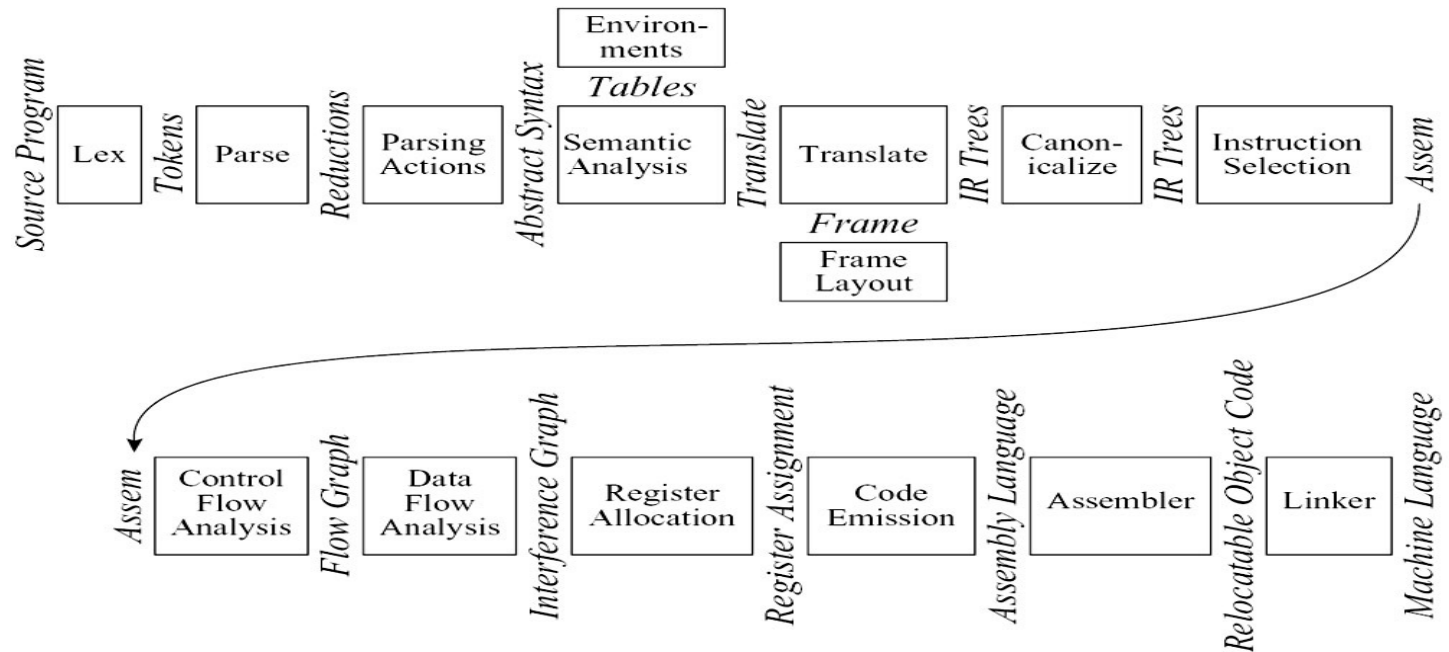
Description of compiler phases

Chapter	Phase	Description
---------	-------	-------------

10	Dataflow Analysis	Gather information about the flow of information through variables of the program For example, <i>liveness analysis</i> calculates the places where each program variable holds a still-needed value (is <i>live</i>)
11	Register Allocation	Choose a register to hold each of the variables and temporary values used by the program; variables not live at the same time can share the same register
12	Code Emission	Replace the temporary names in each machine instruction with machine registers



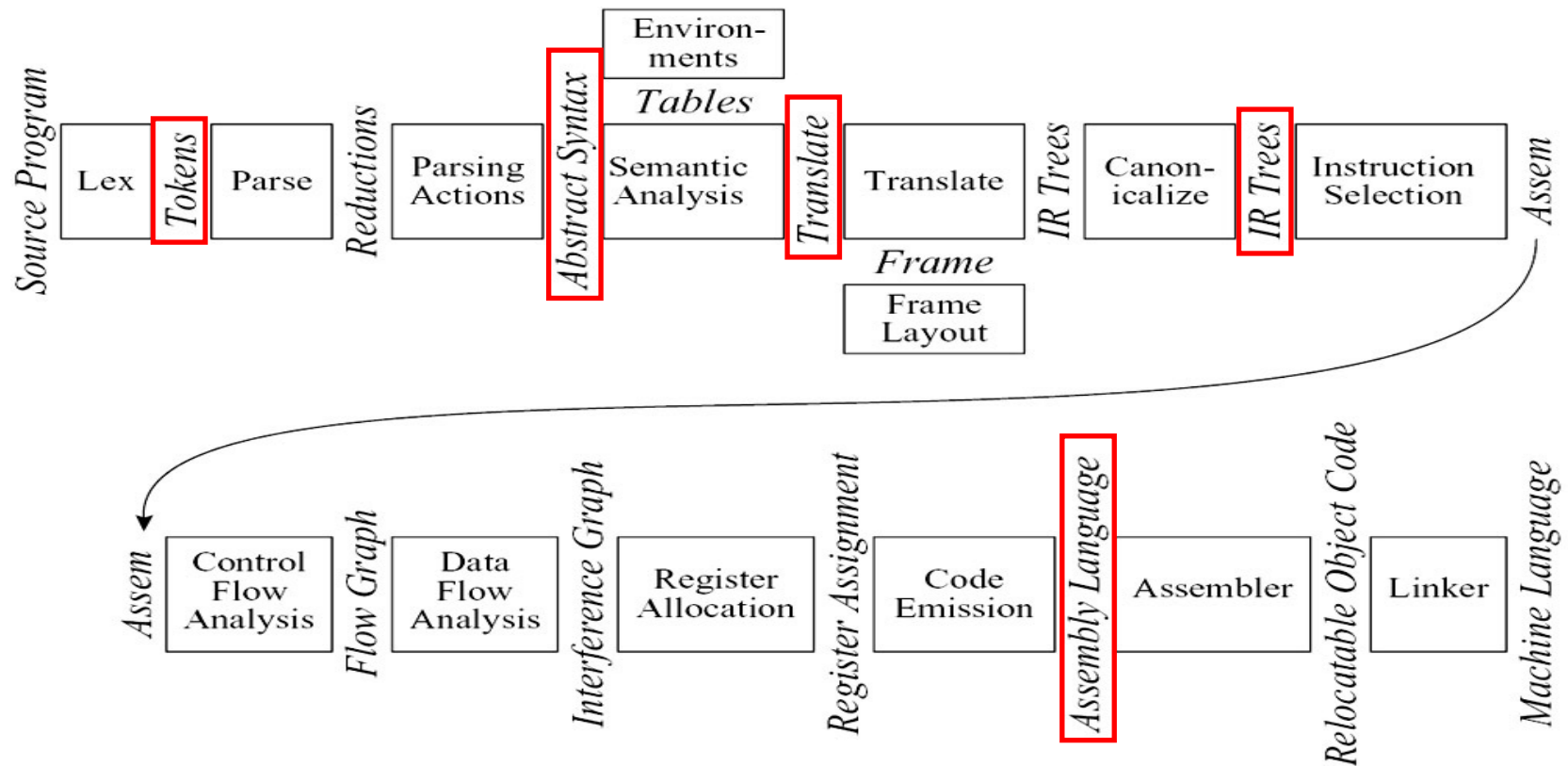
Modules and Interfaces: Modularization



- Several modules maybe combined into one phase
 - Parse, Semantic Analysis, Translate, Canonicalize
 - Instruction Selection maybe combined with Code Emission
- **Simple compilers** may omit the Control Flow Analysis, Data Flow Analysis, and Register Allocation phases

Modules and Interfaces: Interfaces

- **Data structures:** *Abstract Syntax, IR Trees and Assem.*
- **Abstract data types:**
 - A set of functions: the Translate interface
 - A function called by parser: the Token interface



Content

1. INTRODUCTION
2. LEXICAL ANALYSIS
3. PARSING
4. ABSTRACT SYNTAX
5. SEMANTIC ANALYSIS
6. ACTIVATION RECORD
7. TRANSLATING INTO INTERMEDIATE CODE
8. BASIC BLOCKS AND TRACES
9. INSTRUCTION SELECTION
10. LIVENESS ANALYSIS
11. REGISTER ALLOCATION
13. GARBAGE COLLECTION
14. OBJECT-ORIENTED LANGUAGES
18. LOOP OPTIMIZATIONS

Content

1. INTRODUCTION
2. LEXICAL ANALYSIS
3. PARSING
4. ABSTRACT SYNTAX
5. SEMANTIC ANALYSIS
6. ACTIVATION RECORD
7. TRANSLATING INTO INTERMEDIATE CODE
8. BASIC BLOCKS AND TRACES
9. INSTRUCTION SELECTION
10. LIVENESS ANALYSIS
11. REGISTER ALLOCATION
13. GARBAGE COLLECTION
14. OBJECT-ORIENTED LANGUAGES
18. LOOP OPTIMIZATIONS

1.2 Tools and Software

Tools and Software

Two of the most useful abstractions:

- (1) **Regular expressions** for lexical analysis.
- (2) **Context-free grammars** for parsing

Two tools for compiling:

- (2) **Lex** converts a declarative specification into a lexical analysis program
- (2) **Yacc** converts a grammar into a parsing program

The programming project in the textbook can be compiled using any ANSI-standard C compiler, along with *Lex* and *Yacc*.

Labs

To Be Continued